

AI Master Project 7기

최종 발표

과제명 MailTaskAgent · 메일 기반 개인 업무관리 Agent

멘 티 백준현, 09285

멘 토 이유경

발표 영상 10분 이내 | 시연 영상 별도 제출 5분 이내

1. 문제 정의 (Vision / Problem)

어떤 문제를 해결하고자 했는가?

같은 업무 요청이 여러 메일과 Thread에 흩어져 도착합니다. 무엇이 새 업무이고 무엇이 기존 업무의 변경인지 사람이 매번 과거 메일을 다시 찾아 판단해야 합니다.

왜 이 문제가 중요한가? (비즈니스 임팩트)

놓치면 기한을 넘기고, 잘못 붙이면 엉뚱한 업무의 상태가 바뀝니다. 개인의 활성 업무가 수십 건이면 이 판단만으로 하루에 수십 번 문맥을 바꿔야 합니다.

2. 핵심 기능 (Core Features)

무엇을 만들었는가?

메일을 Task로 만들고 기존 업무와 연결한 뒤 다음 Action까지 제안하는 단일 Agent 시스템입니다. 위험한 변경은 사용자 확인으로 넘깁니다.

에이전트 구성:

Mail Analyzer [의미·Intent 구조화] → Context Agent [가설 생성] → Context Agent [가설 평가] → Python Guard [안전 검증] → Reply Agent [회신·승인 발송]

핵심 기술 스택:

회사 gpt-4.1-mini / SQLite Task Context RAG / Python + Pydantic Guard / Gmail API / Streamlit

핵심 성과 (Key Achievements)

28/28

Action 단계 정확도 (회사 LLM Live 15 Case, 정량적 지표)

15/15

시나리오 통과 (같은 15 Case, 정량적 지표)

53.3%

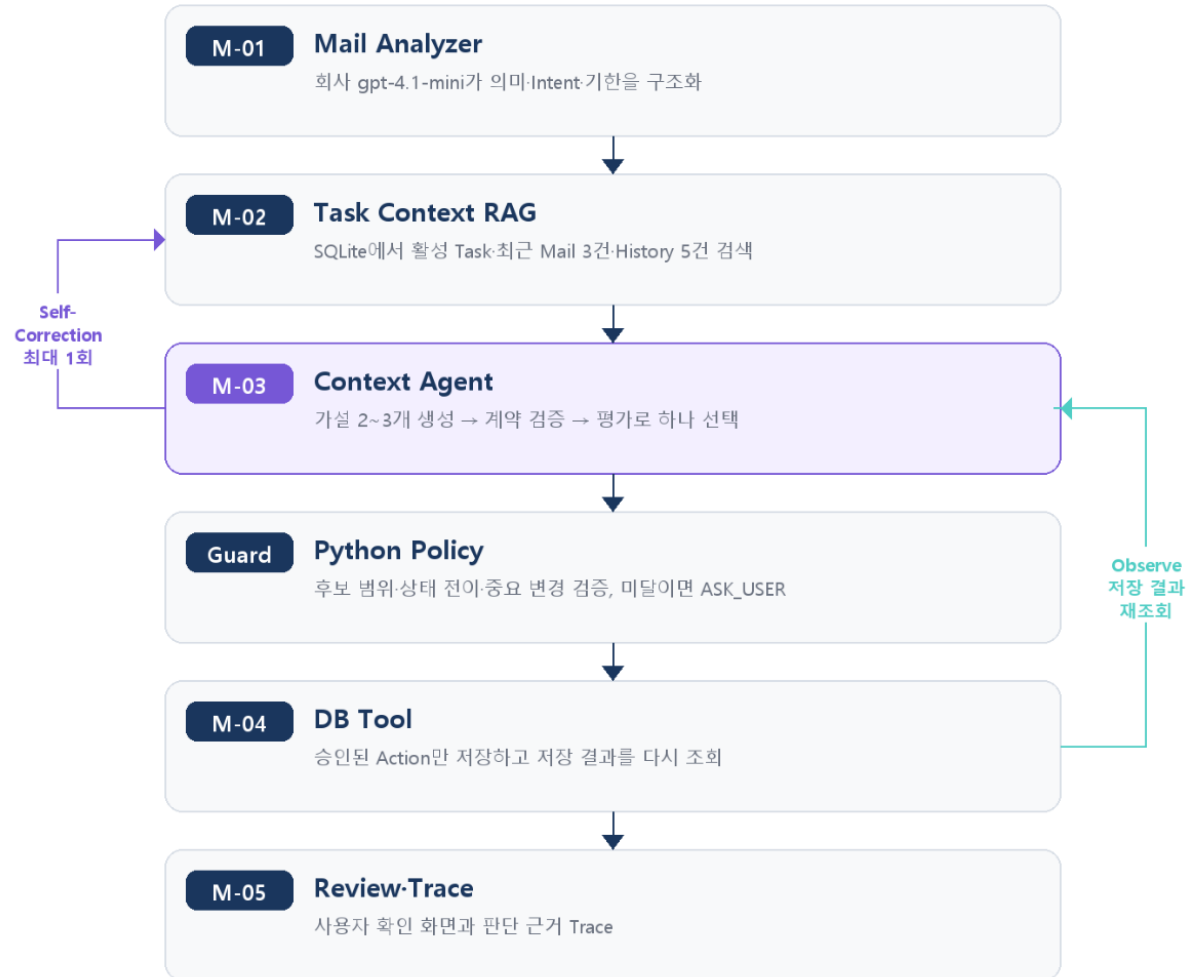
자동 반영 8/15. 나머지 7/15(46.7%)는 Guard가 사용자 확인으로 이관. 정의된 합성 15 Case 기준이며 Mailbox 전체 자동화율이 아닙니다. 사용자 체감 시간은 사용자가 1명이라 순서 효과를 걷어낼 수 없어 미측정으로 남겼습니다.

Key Message

"규칙으로 열거할 수 없는' 이 메일이 어느 업무인가만 Agent가 판단하고, 실행 권한은 Python Guard와 사용자가 가집니다."

Observe → Retrieve → Reason → Act → Observe

제한적 ReAct · 재검색과 재판단은 최대 1회



Vector DB-외부 Embedding 없이 SQLite Structured Retrieval · LLM은 제한만, DB 변경은 Python

기술 아키텍처 고려 사항

왜 이 선택이었는지를 중심으로 설명합니다

01 회사 gpt-4.1-mini + Pydantic Schema

선택 이유:

메일 의미와 Task 관계는 규칙으로 열거할 수 없지만, 출력은 고정 Schema여야 합니다. 사내 승인 모델이라 외부 반출이 없습니다.

핵심 활용:

관계·Action을 계약으로 받고 위반은 재시도로 재생성.

02 SQLite Task Context RAG

선택 이유:

찾을 대상이 외부 문서가 아니라 내 활성 Task·최근 Mail·History·과거 사용자 결정입니다. Vector DB 구성 비용이 더 컸습니다.

핵심 성과:

제한 top-k 검색으로 Live 15 Case Action 28/28.

03 Python + Pydantic Safety Guard

선택 이유:

LLM이 DB를 직접 바꾸면 틀렸을 때 비용이 업무 데이터 오변경입니다. 상태가 닫혀 있어 LangGraph 없이 충분했습니다.

구현 포인트:

후보 범위·Intent·상태 전이 검증, 완료·취소는 승인.

핵심 기술 난제 (Hurdle)

이미 Agent를 쓰고 있었지만, 그 Agent가 관계 하나·대상 하나·Action 하나에 설명 한 줄만 돌려줬습니다. 모델이 실제로 후보를 건주고 골랐는지, 결론을 먼저 정하고 그럴듯한 설명을 붙였는지 밖에서 구분할 방법이 없었습니다. 멘토 피드백도 매번 "Rule Base처럼 보인다"였습니다.

엔지니어링 접근 방법

어떻게 해결했는가? (설계 결정)

가설 생성과 평가를 서로 다른 LLM 호출로 분리했습니다. 생성 단계는 후보 2~3개를 근거와 함께 내고 점수를 매기지 않고, Python이 후보 ID와 관계 계약을 검증한 뒤, 평가 단계가 지지도를 매겨 하나를 고릅니다. 상위 두 지지도의 차이는 Python이 계산합니다.

핵심 알고리즘 / 아키텍처 결정

Bounded Multi-Hypothesis Deliberation — Tree of Thoughts(Yao et al., NeurIPS 2023)의 후보 생성·평가 분리를 참고하되 후보 2~3개·평가 1회로 제한했습니다. Full Tree of Thoughts 구현이 아닙니다. 그 판단을 SQLite Task Context RAG와 제한적 ReAct-style Self-Correction이 Observe → Retrieve → Reason → Act → Observe로 감쌉니다.

왜 이 접근이 필요했는가?

매일 한 건을 업무로 옮기는 판단은 단계가 깊지 않은 대신 틀렸을 때 비용이 업무 데이터 오변경입니다. 그래서 넓게 펼치는 대신 후보를 제한하고, 결정론 Guard와 사용자 승인으로 막았습니다.

다음 단계 (Next Step)

Gmail 검증 결과를 바탕으로 Outlook Graph Adapter와 사내 인증·운영 서버로 전환하고, 검색을 어휘 겹침 너머로 넓히는 것이 다음 과제입니다.

결과 및 성과 (수치로 제시)

100%

Action 단계 정확도

회사 LLM Live 15 Case에서 28/28. 속고 On·Off 동일하며 소요시간만 약 30% 증가

3/3

재현되는 개선 1건

유사 Task 2건 구분 Case에서 속고 On은 3회 모두 사용자 확인, Off는 3회 모두 오연결

0건

계약 위반 판단 유실

검증을 재시도 루프 밖에 두어 실제 Gmail 35건 재생에서 4건 유실 → 루프 안으로 옮겨 0건